

AIML

Python Basics for Data Science

A plain-English study companion with worked examples

- Read alongside the slides

How to use this companion. This document walks beside the lecture slides. The slides show the code and facts; this companion explains the *why*. Five coloured boxes mark five kinds of help: a **blue intuition box** for the plain idea; an **amber everyday box** for a real-life analogy before the math; a **green worked box** for a fully solved example; a **red watch-out box** for the common trap; and a **purple takeaway box** for the one line to remember.

Contents

1	What is Python?	3
1.1	Interpreted vs. compiled	3
2	Setting up your environment	4
3	Variables	5
4	Data types	7
5	Operators	9
5.1	Arithmetic operators	9
5.2	Comparison operators	9
5.3	Logical operators	9
6	User input with <code>input()</code>	11
7	Strings and string slicing	12
8	Data structures	14
8.1	List	14
8.2	Tuple	14
8.3	Set	15
8.4	Dictionary	15
9	Control flow: <code>if</code>, <code>elif</code>, <code>else</code>	17
10	Loops	18
10.1	The <code>for</code> loop	18
10.2	The <code>while</code> loop	18
10.3	<code>break</code> and <code>continue</code>	19
11	Functions	21

12 File handling	23
12.1 Opening and closing a file	23
Glossary & symbols	24

1 What is Python?

Python is a language your computer can read and run line by line. It was designed to look like plain English, so beginners can learn it quickly. It is used in web apps, automation, and — most importantly for us — data science.

The intuition

A programming language is a set of instructions you write for a computer. Python uses simple words such as `if`, `for`, and `print`. The computer reads one line at a time and acts on it straight away. This style is called *interpreted*.

★ Everyday picture

Think of a recipe. A recipe is a list of steps a cook follows one by one. Python is like writing recipes for a very fast, very obedient cook (your computer) that never gets tired. If you make a spelling mistake in step 3, the cook stops and tells you exactly where the problem is.

1.1 Interpreted vs. compiled

When you write Python, a program called the *interpreter* reads your code and runs it immediately. No extra step is needed.

Other languages (like C++) use a *compiler*: it reads the whole program first, translates it into machine code, and then runs the translated version. The compiler is faster at run time but slower to start.

	Interpreter (Python)	Compiler (C++)
Reads code	Line by line	All at once
Runs	Immediately	After translation
Speed of start	Fast	Slower
Error reporting	Stops at the bad line	Reports after whole scan

Where this shows up in ML. Nearly every machine-learning library (NumPy, pandas, TensorFlow, PyTorch) is written for Python. The interpreter lets data scientists try ideas and see results instantly — no compile-and-wait cycle.

✓ Key takeaway

Python is interpreted: it runs your code one line at a time, reports errors immediately, and needs no compilation step.

2 Setting up your environment

You write and run Python inside a tool called an *IDE* (Integrated Development Environment). An IDE is a program that combines a code editor, a way to run code, and a way to see errors — all in one window.

The intuition

An IDE is like a workshop. A plain text editor is just a bench with no tools. An IDE gives you the bench *plus* a saw, a drill, and a tape measure — all within arm's reach.

For data science the most popular choice is **Anaconda** plus **Jupyter Notebook**.

- **Anaconda** is a bundle. It installs Python plus the most-used data science libraries (NumPy, pandas, matplotlib, etc.) in one go.
- **Jupyter Notebook** is a web page that lives on your own computer. You type code in a cell, press **Shift+Enter**, and see the output right below. It also lets you mix code, text, and charts in one document.

Where this shows up in ML. Jupyter notebooks are the standard way to explore data, plot results, and share findings. Almost every online data-science tutorial uses one.

Key takeaway

Install Anaconda once; it gives you Python, Jupyter, and all key libraries in a single step.

3 Variables

A *variable* is a named box that holds a value. You give the box a name, put something in it, and use that name later.

The intuition

Picture a labelled sticky note on a jar. The label is the variable name; what's inside the jar is the value. You can swap out what's in the jar at any time — that's why it's called a *variable*.

★ Everyday picture

In a school register, each student gets a roll number. The roll number is the variable name; the student's score is the value. When the score changes, the roll number stays the same.

You create a variable with the = sign:

```
profit_q1 = 50000
profit_q2 = 75000
name      = "Alice"
is_passed = True
```

No special keyword is needed. Python figures out the type from the value you give.

Worked example: quarterly profit total

The problem: Q1 profit = Rs. 50,000; Q2 = Rs. 75,000; Q3 = Rs. 60,000; Q4 = Rs. 65,000. Find the total.

Step 1. Store each value in a variable.

```
profit_q1 = 50000
profit_q2 = 75000
profit_q3 = 60000
profit_q4 = 65000
```

Step 2. Add them together.

```
total = profit_q1 + profit_q2 + profit_q3 + profit_q4
```

Step 3. Print the result.

```
print("Total profit:", total)
```

Output: Total profit: 250000

Step 4. Sense-check. Four values around 60,000 each. $4 \times 62500 = 250000$. **Correct.**

✗ Watch out

Variable names are case-sensitive. `Name` and `name` are two different variables. Mixing them up causes a `NameError`.

Where this shows up in ML. In a model, every weight and bias is stored as a variable (or a tensor — a multi-value variable). Training means updating those variables to reduce error.

✓ Key takeaway

A variable is a named box. Use = to put something in it. Names are case-sensitive.

4 Data types

Python labels every value with a *data type*. The type tells Python what operations are allowed on that value. Use `type(x)` to check.

The intuition

Think of types like physical containers. A *glass* holds liquid; a *box* holds solid objects. You cannot pour water into a box. Similarly, you cannot subtract one name from another name — strings don't support subtraction.

The main types you will use:

Type	Stores	Example
<code>int</code>	Whole numbers	50000
<code>float</code>	Decimal numbers	3.14
<code>str</code>	Text (characters)	"Alice"
<code>bool</code>	True or False	True
<code>list</code>	Ordered, changeable set	[1, 2, 3]
<code>tuple</code>	Ordered, fixed set	(1, 2, 3)
<code>dict</code>	Key-value pairs	{"a": 1}
<code>set</code>	Unique, unordered items	{1, 2, 3}

Worked example: checking the type of each profit variable

We have `profit_q1 = 50000` and `name = "Alice"`. What type is each?

Step 1. Run `type()` on each.

```
print(type(profit_q1))    # <class 'int'>
print(type(3.14))        # <class 'float'>
print(type("Alice"))     # <class 'str'>
print(type(True))        # <class 'bool'>
```

Step 2. Read the output. `int` means a whole number; `str` means text.

Step 3. Sense-check. Profit is a whole number of rupees, so `int` is correct. A name is text, so `str` is correct. **Both make sense.**

Worked example: password length check

`Password = "Secure123"`. Is the length at least 8?

Step 1. Store the password.

```
password = "Secure123"
```

Step 2. Find the length with `len()`.

```
length = len(password)
print(length)    # 9
```

Step 3. Check the condition.

```
print(length >= 8)    # True
```

Step 4. Sense-check. "Secure123" has 9 characters. $9 \geq 8$ is **True**. Correct.

Where this shows up in ML. Every feature in a dataset has a type. A model that expects

a number will break if it receives a string. Fixing type errors is one of the most common data-cleaning steps.

✓ Key takeaway

Use `type(x)` to check a value's type. Types decide which operations are allowed.

5 Operators

An *operator* is a symbol that performs an action on one or more values.

The intuition

Operators are like the buttons on a calculator. The `+` button adds; the `>` button compares. Python has buttons for maths, comparison, and logic.

5.1 Arithmetic operators

Operator	Meaning	Example
<code>+</code>	Add	<code>5 + 3 = 8</code>
<code>-</code>	Subtract	<code>5 - 3 = 2</code>
<code>*</code>	Multiply	<code>5 * 3 = 15</code>
<code>/</code>	Divide (float)	<code>5 / 2 = 2.5</code>
<code>//</code>	Divide (floor)	<code>5 // 2 = 2</code>
<code>%</code>	Remainder	<code>5 % 2 = 1</code>
<code>**</code>	Power	<code>2 ** 3 = 8</code>

5.2 Comparison operators

These return True or False: `==`, `!=`, `>`, `<`, `>=`, `<=`.

5.3 Logical operators

`and`, `or`, `not` — combine conditions.

Worked example: notebook change

Notebooks cost Rs. 45 each. A student buys 8 and pays with Rs. 500. How much change does the student receive?

Step 1. Store what we know.

```
price_per_notebook = 45
quantity           = 8
amount_paid        = 500
```

Step 2. Calculate total cost.

```
total_cost = price_per_notebook * quantity
# 45 * 8 = 360
```

Step 3. Calculate change.

```
change = amount_paid - total_cost
# 500 - 360 = 140
print("Change:", change)      # Change: 140
```

Step 4. Sense-check. $45 \times 8 = 360$. $500 - 360 = 140$. Correct.

Worked example: student score string

Name = "Alice", Math = 85, Science = 90. Build the sentence: *"Alice scored 85 in Math and 90 in Science."*

Step 1. Store values.

```
name      = "Alice"  
math      = 85  
science   = 90
```

Step 2. Build the string. Use an *f-string* — place a **f** before the opening quote, then put variable names inside **{}**.

```
sentence = f"{name} scored {math} in Math and {science}  
           } in Science."  
print(sentence)
```

Output: Alice scored 85 in Math and 90 in Science.

Step 3. Sense-check. The numbers appear in the right places. **Correct.**

Note: **+** joins strings in Python, but f-strings are cleaner when mixing text and numbers.

✗ Watch out

You cannot use **-** with strings. `"Alice" - "A"` raises a `TypeError`. Use string methods like `replace()` instead.

Where this shows up in ML. The loss function in a neural network is built from arithmetic operators. For example, mean squared error is: sum the squared differences, then divide by the count — just **+**, **-**, ******, and **/**.

✓ Key takeaway

Python has arithmetic (**+** **-** ***** **/** **//** **%** ******), comparison (**==** **!=** **>** **<** **>=** **<=**), and logical (**and** **or** **not**) operators.

6 User input with `input()`

The `input()` function pauses the program and waits for the user to type something. It always returns a *string*, even if the user typed a number.

The intuition

`input()` is like asking someone a question and waiting for the answer. The answer arrives as text; you then decide what to do with it.

Worked example: BMI calculator

Ask the user for height (m) and weight (kg). Print the BMI. BMI formula: $\text{BMI} = \frac{\text{weight}}{\text{height}^2}$.

Step 1. Ask for height and weight.

```
height = float(input("Enter your height in metres: "))
weight = float(input("Enter your weight in kg: "))
```

We wrap `input()` in `float()` because we need decimal numbers, not text.

Step 2. Calculate BMI.

```
bmi = weight / (height ** 2)
```

Step 3. Print with one decimal place.

```
print(f"Your BMI is {bmi:.1f}")
```

Step 4. Sense-check. For height = 1.75 m and weight = 70 kg: $70/(1.75^2) = 70/3.0625 \approx 22.9$. A healthy range is 18.5–24.9, so 22.9 looks right.

Watch out

`input()` always returns a string. Trying to do maths on it directly gives a `TypeError`. Always convert with `int()` or `float()` first.

Key takeaway

`input()` reads text from the user. Wrap it in `int()` or `float()` before doing arithmetic.

7 Strings and string slicing

A *string* is a sequence of characters. Each character has a position number called an *index*, starting at 0.

The intuition

Think of a string as a row of numbered seats. Seat 0 is the first character. Seat 1 is the second. You can ask for any seat by its number, or for a range of seats all at once.

★ Everyday picture

A train has carriages numbered from 0. To book seats in carriages 2 to 5, you say "from 2 to 5". Python slicing works the same way: `s[2:5]` means "from index 2 up to (but not including) index 5".

Slice syntax: `s[start:stop:step]`

- **start** — where to begin (included).
- **stop** — where to end (not included).
- **step** — how many to skip (default 1).

Worked example: extracting first and last name

`full_name = "Edunet Foundation"`. Extract the first word and the second word.

Step 1. Find the space.

```
full_name = "Edunet Foundation"
space_pos = full_name.index(" ")
# space_pos = 6
```

Step 2. Slice before and after the space.

```
first = full_name[:space_pos]           # "Edunet"
last  = full_name[space_pos + 1:]       # "Foundation"
print(first, last)
```

Step 3. Sense-check. "Edunet" is 6 characters (indices 0–5); the space is at index 6. Correct.

Worked example: every second character

`sentence = "Python programming is fun"`. Print every second character.

Step 1. Use a step of 2.

```
sentence = "Python programming is fun"
print(sentence[::2])
```

Output: Pto rgamn sfn

Step 2. Sense-check. Index 0 = 'P', index 2 = 't', index 4 = 'o' ... matches the output. Correct.

Useful string methods: `.upper()`, `.lower()`, `.replace(old, new)`, `.split(sep)`, `.strip()`, `len()`.

Where this shows up in ML. In natural language processing (NLP), every text input is a string. Tokenising a sentence (splitting it into words) uses `.split()`. Cleaning text uses `.replace()` and `.lower()`.

✓ Key takeaway

Strings are indexed from 0. Use `s[start:stop:step]` to slice out any part.

8 Data structures

A *data structure* is a way to hold many values together under one name. Python has four built-in ones you will use constantly.

8.1 List

A *list* is an ordered, changeable collection. You can add, remove, or replace items.

The intuition

A list is a shopping bag. You can put things in any order, add more items, take things out, and change your mind at any time.

```
fruits = ["apple", "banana", "cherry"]
fruits.append("kiwi")           # add to end
fruits.insert(2, "mango")       # insert at position 2
fruits.remove("banana")         # remove by value
print(fruits[0])                # "apple" (index from 0)
```

Worked example: average age from a list

ages = [23, 45, 18, 34, 60, 50, 27]. Find the average.

Step 1. Use `sum()` and `len()`.

```
ages = [23, 45, 18, 34, 60, 50, 27]
average = sum(ages) / len(ages)
print(f"Average age: {average:.1f}")
```

Step 2. Work it out. $23 + 45 + 18 + 34 + 60 + 50 + 27 = 257$. $257/7 \approx 36.7$.

Step 3. Sense-check. Values range from 18 to 60; 36.7 is in the middle. **Reasonable.**

8.2 Tuple

A *tuple* is like a list but *immutable* — you cannot change it after creation. Use parentheses instead of square brackets.

★ Everyday picture

A tuple is like a laminated card. Once printed, the text is fixed. You can read it and copy it, but you cannot write on it.

```
point = (3, 7)
print(point[0])    # 3
# point[0] = 5     # TypeError: tuples are immutable
```

✗ Watch out

If you need to "change" a tuple, you must create a new one. `new_point = (5, point[1])` replaces only the first value.

Where this shows up in ML. Model shapes are stored as tuples: (`batch_size`, `height`, `width`, `channels`). They are tuples because the shape must not change by accident during training.

8.3 Set

A *set* is an unordered collection of *unique* items. Duplicates are removed automatically.

The intuition

A set is like a bag of stamps where you keep only one of each design. You do not care about the order; you only care whether a design is present or not.

Worked example: workshop attendance overlap

```
workshop1 = {"Alice", "Bob", "Charlie", "David"}
workshop2 = {"Eve", "Bob", "Frank", "David"}
Find who attended both; find everyone combined.
```

Step 1. Intersection (both).

```
both = workshop1 & workshop2
print("Both:", both)      # {'Bob', 'David'}
```

Step 2. Union (anyone).

```
everyone = workshop1 | workshop2
print("All:", everyone)
# {'Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank'}
```

Step 3. Sense-check. Bob and David appear in both lists; the union has 6 unique names. **Correct.**

8.4 Dictionary

A *dictionary* stores key–value pairs. You look up a value by its key, just like a real dictionary.

★ Everyday picture

A dictionary is a phone book. The name is the key; the phone number is the value. You search by name to find the number.

```
student = {"name": "Alice", "age": 20, "grades": [85, 90, 78]}
print(student["name"])      # Alice
student["major"] = "CS"     # add a new key
del student["age"]          # remove a key
```

Worked example: average grade from a dictionary

Dictionary: {"name": "Alice", "grades": [85, 90, 78]}. Calculate the average grade.

Step 1. Access the grades list.

```
student = {"name": "Alice", "grades": [85, 90, 78]}
grades = student["grades"]
```

Step 2. Calculate the average.

```
avg = sum(grades) / len(grades)
print(f"Average: {avg:.1f}")
```

Step 3. Work it out. $85 + 90 + 78 = 253$; $253/3 \approx 84.3$.

Step 4. Sense-check. Grades are 78–90; the average should be between them. 84.3 is. **Correct.**

Where this shows up in ML. Model hyperparameters (learning rate, number of layers, batch size) are stored in dictionaries. Libraries like Scikit-learn use dictionaries to pass settings.

✓ **Key takeaway**

Four data structures: **list** (ordered, changeable), **tuple** (ordered, fixed), **set** (unique items, no order), **dict** (key–value pairs).

9 Control flow: if, elif, else

Control flow means choosing which code to run based on a condition.

The intuition

Control flow is a signpost at a crossroads. "If it rains, take an umbrella. Otherwise, wear sunglasses." The program reads the condition and picks a path.

Syntax:

```
if condition_1:
    # run when condition_1 is True
elif condition_2:
    # run when condition_1 is False AND condition_2 is True
else:
    # run when all conditions above are False
```

Worked example: age category

Read a user's age and print: child (0–12), teenager (13–19), adult (20–64), or senior (65+).

Step 1. Read the age.

```
age = int(input("Enter age: "))
```

Step 2. Check each range in order.

```
if age <= 12:
    print("Child")
elif age <= 19:
    print("Teenager")
elif age <= 64:
    print("Adult")
else:
    print("Senior")
```

Step 3. Trace with age = 17. $17 \leq 12$? No. $17 \leq 19$? Yes. Prints "Teenager". Correct.

Step 4. Sense-check. Every age from 0 to any large number is covered. No gap; no overlap.

Note: Python stops at the first **True** branch. Later branches are not checked.

Watch out

Use `==` to compare values, not `=`. `if x = 5:` is a syntax error in Python. `=` assigns; `==` compares.

Where this shows up in ML. Decision trees work exactly like chains of `if` statements. Each node checks one feature and sends the data left or right.

Key takeaway

`if/elif/else` runs different code for different conditions. Python takes the first **True** branch and skips the rest.

10 Loops

A *loop* repeats a block of code. This saves you from writing the same line 100 times.

The intuition

A loop is a merry-go-round. The ride repeats until the operator presses stop. In code, the "stop" is when the condition is no longer true, or when you have visited every item.

10.1 The for loop

A *for* loop visits each item in a sequence one by one.

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
# apple
# banana
# cherry
```

Use `range(n)` to loop a fixed number of times:

```
for i in range(5):      # i goes 0, 1, 2, 3, 4
    print(i)
```

Worked example: grading students from a dictionary

Dictionary of names and scores: {"Alice": 92, "Bob": 75, "Carol": 61}. Print each student's grade (A for 90+, B for 80–89, C for 70–79, D for 60–69, F below 60).

Step 1. Set up the dictionary and loop.

```
scores = {"Alice": 92, "Bob": 75, "Carol": 61}

for name, score in scores.items():
```

Step 2. Assign the grade inside the loop.

```
    if score >= 90:
        grade = "A"
    elif score >= 80:
        grade = "B"
    elif score >= 70:
        grade = "C"
    elif score >= 60:
        grade = "D"
    else:
        grade = "F"
    print(f"{name}: {grade}")
```

Step 3. Expected output: Alice: A, Bob: C, Carol: D.

Step 4. Sense-check. Alice scored 92 (above 90) → A. Bob scored 75 (70–79) → C. Carol scored 61 (60–69) → D. **All correct.**

10.2 The while loop

A *while* loop repeats as long as a condition is `True`.

```
count = 0
while count < 5:
    print(count)
    count += 1      # count = count + 1
```

Worked example: collect numbers until sum exceeds 100

Repeatedly ask the user for a number. Stop when the running total is above 100. Print the list and the total.

Step 1. Set up.

```
numbers = []
total   = 0
```

Step 2. Loop until total > 100.

```
while total <= 100:
    n = int(input("Enter a number: "))
    numbers.append(n)
    total += n
```

Step 3. Print the result.

```
print("Numbers:", numbers)
print("Sum:", total)
```

Step 4. Trace (example run): user enters 40, 35, 30. After 30: total = 105 > 100. Loop stops. Output: Numbers: [40, 35, 30], Sum: 105. **Correct.**

10.3 break and continue

- **break** — stop the loop immediately and jump past it.
- **continue** — skip the rest of this iteration and go to the next one.

Worked example: check for empty strings in a list

List: ["hello", "", "world", "python"]. Report whether any string is empty.

Step 1. Loop and check.

```
items    = ["hello", "", "world", "python"]
found    = False

for item in items:
    if item == "":
        found = True
        break      # stop as soon as we find one
```

Step 2. Report.

```
if found:
    print("List contains empty strings")
else:
    print("No empty strings found")
```

Step 3. Sense-check. The second element is "", so found becomes True and the loop stops. Output: "List contains empty strings". **Correct.**

Where this shows up in ML. The main training loop in deep learning is a **for** loop over *epochs* (passes through the data). Inside each epoch is another loop over *batches*. Early stopping — halting training when the model stops improving — is a **break**.

✓ Key takeaway

for loops visit each item. **while** loops repeat until a condition is false. **break** exits early; **continue** skips to the next iteration.

11 Functions

A *function* is a named block of code that you can run again and again by calling its name.

The intuition

A function is a vending machine. You press a button (call the function), maybe put in coins (pass arguments), and get a snack back (return value). You do not need to know what happens inside.

Syntax:

```
def function_name(parameter1, parameter2):
    # code
    return result
```

Worked example: calculate average

Write a function that takes a list of numbers and returns their average.

Step 1. Define the function.

```
def calculate_average(numbers):
    total    = sum(numbers)
    count    = len(numbers)
    return total / count
```

Step 2. Call it with a list of scores.

```
scores = [72, 85, 91, 68, 79]
avg     = calculate_average(scores)
print(f"Average score: {avg:.1f}")
```

Step 3. Work it out. $72 + 85 + 91 + 68 + 79 = 395$; $395/5 = 79.0$.

Step 4. Sense-check. Values range from 68 to 91; 79.0 sits in the middle. **Correct.**

Note: putting this logic inside a function means you can call it on any list — not just this one.

Worked example: multiply a list by a tax rate

Write `multiply_list(prices, multiplier)` and use it to add 20% tax.

Step 1. Define the function.

```
def multiply_list(prices, multiplier):
    return [p * multiplier for p in prices]
```

This uses a *list comprehension*: it loops over every price `p` and builds a new list with each one multiplied.

Step 2. Call it.

```
prices      = [100, 250, 80, 175]
with_tax    = multiply_list(prices, 1.2)
print(with_tax)
# [120.0, 300.0, 96.0, 210.0]
```

Step 3. Sense-check. $100 \times 1.2 = 120.0$. $250 \times 1.2 = 300.0$. **Correct.**

X Watch out

A variable you create inside a function is *local*: it disappears when the function ends. If you need the value outside, use **return**.

Where this shows up in ML. Every step in an ML pipeline — load data, clean data, train model, evaluate model — is written as a function. This makes it easy to swap out one step without breaking the others.

✓ Key takeaway

Define a function with **def**. Pass inputs as parameters; get outputs with **return**. Functions let you reuse code without copying it.

12 File handling

File handling means reading from and writing to files stored on disk. This lets your program save results and load data between runs.

The intuition

File handling is like a filing cabinet. To read a file, you open the drawer, take out the folder, read it, and close the drawer. To write, you open the drawer, put in a new page, and close it. Always close the drawer!

12.1 Opening and closing a file

Use `open(filename, mode)`. Modes:

Mode	Meaning
"r"	Read (default). File must exist.
"w"	Write. Creates the file; erases existing content.
"a"	Append. Creates the file; adds to existing content.
"r+"	Read and write.

Always close the file with `file.close()`, or use a `with` block (better):

```
with open("data.txt", "r") as f:
    content = f.read()
# File closes automatically here
```

The `with` keyword closes the file for you, even if an error happens inside the block.

Worked example: write grades to a file

Write student names and grades to `grades.txt`. Data: `[("Alice", "A"), ("Bob", "C"), ("Carol", "D")]`.

Step 1. Open in write mode and loop.

```
students = [("Alice", "A"), ("Bob", "C"), ("Carol", "D")]

with open("grades.txt", "w") as f:
    for name, grade in students:
        f.write(f"{name},{grade}\n")
```

"\n" moves to the next line after each entry.

Step 2. Check the file. Opening `grades.txt` shows:

```
Alice,A
Bob,C
Carol,D
```

Step 3. Sense-check. Three students, three lines. Each line has a name and grade separated by a comma. **Correct.**

Worked example: read a CSV file and total the prices

File `products.csv` format: `ProductName,Price`. Read it and print the total cost.

Step 1. Open and read line by line.

```
total = 0.0

with open("products.csv", "r") as f:
    next(f)                # skip the header
    line
    for line in f:
        line = line.strip()    # remove newline
        parts = line.split(",") # split at comma
        price = float(parts[1]) # second column is
                                price
        total += price
```

Step 2. Print the total.

```
print(f"Total cost: Rs. {total:.2f}")
```

Step 3. Sense-check. We skip the header, split each line, take the second element (price), convert to float, and add. If the file has 4 products at Rs. 50 each, total = Rs. 200. **Logic is sound.**

Note: for larger CSV files, the `csv` module or `pandas` is faster and safer — but this basic approach works for small files.

✗ Watch out

"w" mode *erases* the file before writing. If you want to add to an existing file, use "a" (append) mode.

Where this shows up in ML. Every ML project loads data from files: CSV files for tabular data, text files for NLP corpora, JSON for model configs. Knowing file I/O lets you work with real datasets instead of toy variables.

✓ Key takeaway

Use `with open(filename, mode) as f:` to handle files. Mode "r" reads; "w" writes (erasing first); "a" appends. The `with` block closes the file automatically.

Glossary & symbols**Key terms.**

variable	a named box that holds a value; created with <code>=</code> .
data type	the kind of value a variable holds (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code> , etc.).
interpreter	the program that reads and runs Python code one line at a time.
IDE	Integrated Development Environment; a tool that combines an editor, a runner, and a debugger.
string	a sequence of characters; written in quotes.
index	the position of an item, starting at 0.
slicing	taking a sub-sequence with <code>s[start:stop:step]</code> .

list	an ordered, changeable collection of values.
tuple	an ordered, unchangeable collection.
set	an unordered collection of unique items.
dictionary	a collection of key–value pairs.
control flow	choosing which code to run with if/elif/else .
loop	repeating code; for visits items one by one; while repeats while a condition is true.
function	a named, reusable block of code defined with def .
parameter	a variable that receives a value when a function is called.
return	the value a function sends back to the caller.
file handling	reading from and writing to files on disk using open() .
f-string	a string prefixed with f that embeds variable values inside {} .

Symbols and syntax at a glance.

Symbol / syntax	Meaning
=	Assign a value to a variable
==	Compare two values; returns True or False
!=	Not equal
**	Power: $2**3 = 8$
//	Floor division: $7//2 = 3$
%	Remainder: $7\%2 = 1$
len(x)	Number of items in x
type(x)	The data type of x
int(x)	Convert x to a whole number
float(x)	Convert x to a decimal number
str(x)	Convert x to a string
range(n)	Integers 0, 1, ..., n-1
s[i]	Character at index i in string or list s
s[a:b]	Slice from index a to b -1
def f(p):	Define function f with parameter p
return v	Send value v back from a function
open(f,m)	Open file f in mode m